

GitHub-copilot and RAG

Dr. Shila Jawale

Assistant Professor

Department of Computer Engineering

K. J. Somaiya College of Engineering

Somaiya Vidyavihar University

Topic for Discussion

1. GitHub Copilot

2. RAG

GitHub Copilot

<https://www.youtube.com/watch?v=SJqGYwRq0uc>



What is GitHub Copilot?

👉 It's an **AI-powered coding assistant** developed by GitHub in collaboration with OpenAI.

It helps you:

- Write code automatically ✨
- Suggest entire functions
- Complete lines as you type

GitHub Copilot with VS Code – From Instruction to Code



1. Integration of GitHub Copilot with VS Code

1 Open VS Code

Download and install Visual Studio Code.



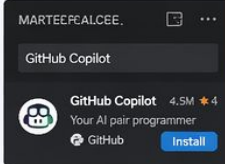
2 Go to Extensions

Click on the Extensions icon in the Activity Bar.



3 Search GitHub Copilot

Search for "GitHub Copilot" in the Extensions Marketplace.



4 Install Extension

Click Install and reload VS Code if prompted.

Install

5 Sign in to GitHub

Sign in with your GitHub account and activate Copilot.

✓ GitHub Copilot is now active!

✓ GitHub Copilot is successfully integrated with VS Code!

2. Instruction Written for Generating Code

In the code editor, I wrote the following instruction (as a comment):

```
# Write a Python function to find the factorial of a number  
# using recursion
```

3. Corresponding Code Suggested by GitHub Copilot

```
factorial.py ×  
1 # Write a Python function to find the factorial of a number  
2 # using recursion  
3  
4 def factorial(n):  
5     """Return the factorial of a non-negative integer n."""  
6     if n < 0:  
7         raise ValueError("Factorial is not defined for negative numbers")  
8     if n == 0:  
9         return 1  
10    return n * factorial(n - 1)  
11
```

✨ Copilot suggestion · Tab to accept

i Start writing a comment or partial code → Copilot understands the context
→ suggests the complete code → Press **Tab** to accept.

RAG(Retrieval Augmented Generation)

Introduction to Large Language Model(LLMs)

What is Large Language Models (LLMs)



Simple Definition

- **LLM (Large Language Model)** = AI trained on massive amounts of text data
- Learns patterns of language, meaning and context
- Can generate human-like responses



How It Works (In One Line)

- ➔ Predicts the next word (or token) based on previous words



Think of It Like

- A super-smart autocomplete system
- Similar to your phone typing suggestions — but **much more powerful**



Key Insight: LLMs *don't* 'know' facts like humans — they predict what comes next based on **patterns**



AI Model

....

Input:

The capital of India is... |



Output:

New Delhi



Key Insight:

LLMs *don't* "know" facts like humans — they predict what comes next based on **patterns**

Popular Large Language Models (LLMs)



ChatGPT

- Developed by **OpenAI**
- Strong in conversation, coding, and general tasks
- Widely used by students & professionals



Gemini

- Developed by **Google**
- Integrated with **Google Search & Workspace**
- Strong in multimodal (text + image + video)



Claude

- Developed by **Anthropic**
- Focus on **safety** and long document understanding
- Good for research and analysis tasks



LLaMA

- Developed by **Meta**
- Open-source / customizable
- Popular for research and building custom AI systems

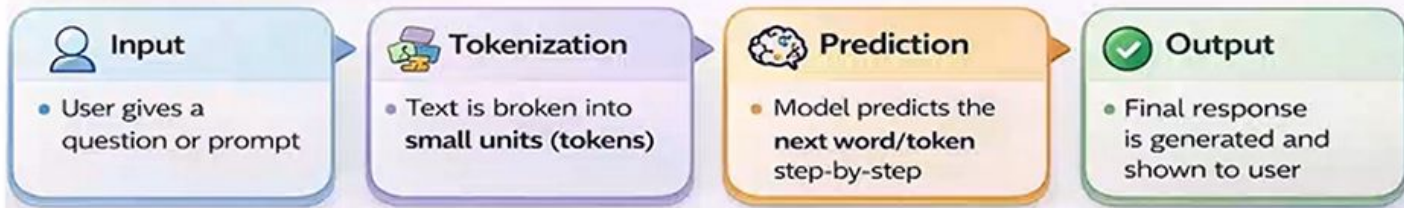
- **Different LLMs have different strengths- choose based on your use case**
- **All these models look similar from outside, but internally they are trained differently and optimized for different tasks.**

How LLM works?

Step-by-Step Flow

⇒ Input → Tokenization → Prediction → Output

! What Happens Internally?



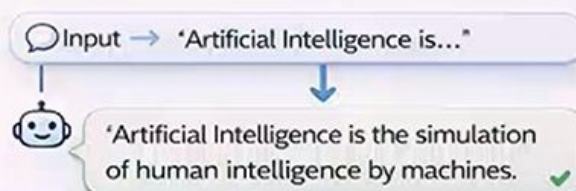
💡 Simple Analogy

⇒ Works like a smart autocomplete system

- Similar to phone typing suggestions
- But much more context-aware and intelligent

⚙️ Quick Example

💬 Input: → 'Artificial Intelligence is...'



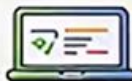
LLMs generate responses one word at a time- not all at once

Strengths of LLMs



1. Natural Language Understanding

- Understands human language in a conversational way
- Can answer questions, explain concepts, and hold discussions
- Useful for teaching, tutoring, and support systems



2. Coding Help

- Generates **code snippets**
- Explains programming logic
- Helps debug errors
- Supports learning in languages like Python, Java, C++



3. Summarization

- Converts long content into short **key points**
- Useful for:
 - Research papers
 - Lecture notes



4. Content Generation

- Writes emails, reports, **essays**, and study notes
- Creates **quiz questions**
- Drafts project ideas
- Assists in **academic writing**

LLMs boost productivity by helping humans learn, create, and solve faster

Demo (Screenshots)



Live Prompt to Ask

Input :

"Explain OSI model in simple terms."



What This Demonstrates



Structured Output

- Organized in steps
- Clear numbering
- Logical flow



Clarity

- Technical concept simplified
- Easy analogy used
- Beginner-friendly explanation



Sample LLM Response



OSI Model – Simple Explanation

The **OSI** (Open Systems Interconnection) model describes how data moves between computers in a network.

Think of it like **sending a package through different checkpoints**, where each checkpoint handles a specific task.

> 7 Layers of OSI Model:

-  **Physical Layer** – Sends signals through cables/wireless
-  **Data Link Layer** – Transfers data between nearby devices
-  **Network Layer** – Decides best route for data
-  **Transport Layer** – Ensures reliable delivery
-  **Session Layer** – Manages communication sessions
-  **Presentation Layer** – Formats/encrypts data
-  **Application Layer** – Interfaces with users (browser, email)

Key Takeaways



- Use LLM as assistants, not unquestioned authorities.
- LLM => **Powerful** but not perfect

LLM Limitations- Hallucination Problem

😬 What is Hallucination in LLMs?

- LLM generates an answer that sounds **confident and believable**
- But the information is actually **incorrect or completely made up**

🧪 Example

- ❌ **Fake Research Citations**

LLM may generate:

- Non-existent journal papers
- Fake author names
- Incorrect publication years

Even though response looks academic and trustworthy

🎯 Why It Matters

- Dangerous in research
- Misleads students
- Can spread misinformation

**Confidence in wording does not
guarantee correctness**

LLM Limitations- No Real Time Knowledge

🤔 Knowledge Limitations-

- Most LLMs learn from data used during training
- They may not know:
 - Latest news
 - Recent discoveries
 - New regulations
 - Current events

🌀 Example

- Ask -
“Explain our college syllabus.”
=> LLM may miss latest updates.

🔍 Reason

- 👉 Knowledge is limited to a **training cutoff date**

👉 ***Without external search, LLM knowledge can become outdated.***

Why LLM limitations matters?

Key Message

-  **LLM Alone = Not reliable for critical use**
- Asking identical prompt multiple times may produce:
 - ✓ Different wording
 - ✓ Different examples
 - ✓ Sometimes different conclusions

Why?

- Can hallucinate
- Lacks latest updates
- Cannot access private knowledge
- Hard to verify sources
- May be inconsistent

 **Human verification is essential when using LLMs for research, education, and decision-making.**

Why Do We Need Something Beyond LLMs?- Key Needs of RAG



Accuracy

- Reduce **wrong** / hallucinated answers



Updated Data

- Access latest information



Custom Knowledge

- College notes
- Research papers
- Internal documents



Key Needs



Accuracy

- Reduce wrong / hallucinated answers



With Open Book (RAG)

- Refers to correct material
- Gives accurate answers



LLMs alone cannot meet real-world accuracy and data needs

RAG = Retrieval-Augmented Generation



Core Idea 🖐️ Retrieve data ➡️ Then generate answer

What Happens?

First:



Find relevant information



Then:

Use LLM to generate answer based on that info



Simple Analogy



Student Exam Analogy



Without Book (LLM)

- Relies only on memory
- May forget or guess



VS.



With Open Book (RAG)

- Refers to correct material
- Gives accurate answers



RAG = Open-book intelligence

RAG FLOW (Simple)

Step-by-Step Process



In One Line



Search first, then answer



User



Search



Retrieve



LLM



Answer

RAG Example



Question, "Explain our college AI syllabus"



Using LLM Only



Generic answer



Not specific to your college



Using RAG



Uses actual syllabus



Provides accurate,
relevant explanation



RAG makes AI domain-specific and useful

Key Benefit-Why RAG is Powerful



Core Message: 📌 *Grounded, Accurate, Contextual AI*



What This Means



Grounded

- Responses are based on real documents & trusted data sources
- *Not just model memory*



Accurate

- Reduces hallucinations
- Improves reliability for decision-making



Contextual

- Tailored to your domain (college, enterprise, research, etc.)
- Produces relevant and specific answers



Final Takeaway



RAG transforms LLMs

from generic tools ➡ **into reliable, domain-aware systems**

“RAG is the missing layer that makes AI usable in real-world academic and enterprise scenarios.”

RAG- High Level Architecture



How RAG System Works (End-to-End)



Data → Embeddings → Vector DB → Retrieval → LLM → Answer



Explanation (Simple Terms)



Key Insight



RAG combines storage + search + intelligence



RAG combines storage + search + intelligence

INPUT DATA SOURCES

Step-by-Step Process



Goal



Collect all relevant knowledge in one place



Key Insight



Quality of data directly impacts quality of answers

Step-2 Chunking

1 What is Chunking?

Breaking large documents into smaller pieces (chunks)



Chunk 1
Paragraph/Section

Chunk 2
Paragraph/Section

Chunk 3
Paragraph/Section

⋮

Chunk N
Paragraph/Section

2 Why Needed?



LLM cannot process
very large text at once
Limited context window
(e.g., a few thousand tokens)



Smaller chunks improve
search accuracy
More precise and relevant
information retrieval

3 Example

100-page PDF → split into
paragraphs/sections



Chunk 1
Pages 1–2

Chunk 2
Pages 3–5

Chunk 3
Pages 6–8

⋮

Chunk N
Pages 98–100



Key Insight



Better chunking = better retrieval



Step-3 Embedding

- ✓ Convert text into **numerical vectors**
- ✓ Capture meaning and context



Why Important?



Helps system understand semantic similarity



Example

“AI” and “Artificial Intelligence” → **similar vectors**

“AI” →

0	3	0	8	0	1	7	2	2	4	6	6	5	7	2	2
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

“Artificial Intelligence” →

0	3	7	2	0	8	3	4	4	7	4	3
---	---	---	---	---	---	---	---	---	---	---	---



Key Insight



Embeddings enable meaning-based search
(not just keyword search)

Step-4 Vector Database



Where Data is Stored



Stores embeddings for fast retrieval



Popular Tools

-  FAISS
-  Pinecone



Role

- Quickly find similar vectors (relevant content)



Key Insight

- ▶ Vector DB = **Brain memory** for RAG systems



Vector DB = Brain memory for RAG systems

Step-5 Retrieval



What Happens Here?

- 1 System searches vector DB
- 2 Finds most relevant chunks



How It Works

- 1 Compare query embedding with stored embeddings
- 2 Return top matching results



Key Insight



Retrieval ensures answers are grounded in real data.

Step-6 Generation



Final Step

LLM receives:



User query



Retrieved context



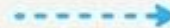
What LLM Does



Combines both



Generates accurate, contextual answer



Generates accurate, contextual answer



Key Insight



LLM answers based on retrieved knowledge—not just memory



Complete Pipeline

👉 User → Search → Retrieve → LLM → Answer



User



Search



Retrieve



LLM



Answer



In One Line



Search + Think + Answer



What This Means



Search → Find relevant data



Think → LLM processes context



Answer → Final response



Final Takeaway



RAG turns LLM into a reliable, knowledge-driven system

“This architecture is what makes AI usable in real-world applications like education, healthcare, and enterprise systems.”

Technology Stack



Python

Core programming language



LangChain

Framework to build RAG pipelines



OpenAI API

For embeddings + LLM responses



**These tools simplify building AI applications
without deep ML knowledge**

DEMO Steps



Step-by-Step Flow



In One Line 🖱️ Upload $\rightarrow$ Process $\rightarrow$ Search $\rightarrow$ Answer

Sample code



Basic RAG Implementation

```
from langchain.document_loaders import PyPDFLoader
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings

# Load PDF
loader = PyPDFLoader("notes.pdf")
docs = loader.load()

# Create vector database
db = FAISS.from_documents(docs, OpenAIEmbeddings())

# Ask question
query = "Explain topic"
docs = db.similarity_search(query)

# Show answer
print(docs[0].page_content)
```



What This Code Does

- ✓ Loads PDF
- ✓ Converts into embeddings
- ✓ Stores in vector DB
- ✓ Retrieves relevant content for query



Key Insight



Even a few lines of code can build a powerful AI system

"This is where theory becomes real—students can build this in a single lab session."

Advanced RAG patterns

Why Advanced RAGs?

! Why Do We Need Advanced RAG Techniques?

🧠 Key Reasons

✓ Improve Accuracy

- Better retrieval → Better answers
- Reduce hallucinations

💡 Handle Complex Queries

- Multi-step questions

💡 Handle Complex Queries

- Multi-step questions
- Ambiguous queries
- Domain-specific problems



💡 Key Insight

- Basic RAG works – but Advanced RAG makes it reliable and scalable

👉 **Advanced RAG makes AI systems more reliable, scalable and intelligent**

Common Techniques



Multi-Query RAG

- Generate multiple variations of a query
- Improves retrieval coverage



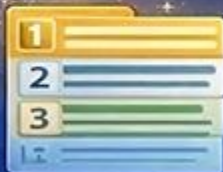
Hybrid Search (Keyword + Vector)

- Combines:
 - Traditional keyword search
 - Semantic (embedding-based) search
- Gives more accurate results



Re-ranking

- Reorders retrieved results
- Keeps most relevant content on top



Re-ranking

- Reorders retrieved results
- Keeps most relevant content on top



Better retrieval = better generation

Agentic RAGs?

What is Agentic RAG?

AI acts like an agent (decision-maker)



What It Does



AI decides:



What to search



Which sources to use



How to combine information



How to answer



Key Insight



Agentic RAG = Autonomous, intelligent retrieval + reasoning



Agentic RAG = Autonomous, intelligent retrieval + reasoning

Example

Complex research question

AI performs multiple searches before answering



Evaluation Parameters

How Do We Measure RAG Performance?



Accuracy Are answers correct?



Relevance Are retrieved documents useful for the query?



Latency How fast does the system respond?

Why It Matters

Balance between:



+



Key Insight



A good RAG system is accurate, relevant, and fast

“Building RAG is not enough—evaluating and improving it is what makes it production-ready.”



Where Can RAG Be Used in Academia?

1. 1. AI Tutor

- Answers questions directly from syllabus & lecture notes
- Provides personalized explanations
- Available 24/7 for students



2. 2. Research Assistant

- Summarizes research papers & journals
- Extracts key insights quickly
- Helps in literature review



3. 3. Assignment Helper

- Provides context-based answers
- Uses course material instead of generic info
- Improves learning quality (not just copying)

4. 4. College Knowledge Bot

- Answers questions about:

- 📅 Policies
- 📖 Curriculum
- 📅 Academic schedules



Key Insight



RAG enables domain-specific, reliable AI for education systems



RAG enables domain-specific, reliable AI for education systems

Real-World Examples of RAG

(Retrieval-Augmented Generation)

Customer Support Chatbots



E.g. "How to reset my Wi-Fi?"
Generates troubleshooting steps.

Healthcare Assistance



E.g. "Patient has chest pain."
Gives clinical guidelines.

Legal Document Analysis



E.g. "Cases on data privacy."
Summarizes key rulings.

Legal Document Analysis



E.g. "Cases on data privacy."
Provides precise info.

Enterprise Knowledge



E.g. "What's the leave policy?"
Provides precise info.

E-Commerce Assistance



E.g. "Best gaming laptop under 80K."
Recommends top options.

Educational Tools



E.g. "Explain SQL injection."
Gives detailed explanation.

Financial Advisory



E.g. "Invest in mutual funds?"
Offers market insights.

News & Research



E.g. "Latest in cybersecurity."
Summarizes the updates.

How RAG Works:



Retrieve
Find Info.



Augment
Add to Prompt



Generate
Create Answer

Final Takeaways



Core Concepts



LLM = Brain



RAG = Brain + Books



Future = Domain-Specific AI Systems



Thank you.